

Contents

Getting Started with CSQL.jl	1
Introduction	1
Setup	1
Creating Causal Triples	2
Building the Database	2
Basic Queries	3
Atlas Statistics	3
Causal Backbone	3
Causal Hubs	3
Downstream Effects	4
Upstream Causes	4
Custom SQL Queries	4
Summary	5

Getting Started with CSQL.jl

- Introduction
- Setup
- Creating Causal Triples
- Building the Database
- Basic Queries
 - Atlas Statistics
 - Causal Backbone
 - Causal Hubs
 - Downstream Effects
 - Upstream Causes
- Custom SQL Queries
- Summary

Introduction

CSQL.jl implements the *Csql* causal database framework (arXiv:2601.08109) in Julia. It builds SQL-queryable causal databases from extracted causal claims — triples of the form *(subject, relation, object)* — and provides tools for querying, intervention analysis, and multi-source merging.

This vignette walks through the basics: creating a builder, adding causal triples, building the database, and running your first queries.

Setup

`using CSQL`

Creating Causal Triples

A *causal triple* represents a single claim: **A** *relation* **B**. For example, “Vaccination reduces Susceptibility”.

CSQL.jl uses an `AtlasBuilder` to accumulate triples before writing them to a database.

```
builder = AtlasBuilder()

# Add epidemiological causal claims
add_triple!(builder, "Pathogen transmissibility", "increases", "Transmission rate";
             doc_id="smith2023", score=0.92)

add_triple!(builder, "Contact rate", "increases", "Transmission rate";
             doc_id="jones2024", score=0.87)

add_triple!(builder, "Vaccination coverage", "reduces", "Host susceptibility";
             doc_id="chen2023", score=0.85)

add_triple!(builder, "Vaccination coverage", "increases", "Population immunity";
             doc_id="chen2023", score=0.91)

add_triple!(builder, "Host susceptibility", "increases", "Transmission rate";
             doc_id="smith2023", score=0.80)

add_triple!(builder, "Transmission rate", "increases", "Outbreak severity";
             doc_id="smith2023", score=0.95)
```

Each triple is automatically:

- **Canonicalized** — labels are lowercased and normalized
- **Relation-typed** — free-text relations like “increases” are mapped to standard types (INCREASES, REDUCES, etc.)
- **Scored** — each claim carries a confidence score (0–1)

Building the Database

Connect to an in-memory SQLite database and build the atlas:

```
csql = connect_csql()
build!(builder, csql.db)

SQLite.DB(":memory:")
```

The `build!` step:

1. Computes stable hash IDs for each node and edge
2. Aggregates duplicate edges (same source→target→relation) across documents

3. Calculates degree statistics, polarity mass, and controversy scores
4. Detects strongly connected components (feedback loops)
5. Writes four tables: `atlas_nodes`, `atlas_edges`, `atlas_edge_support`, `atlas_scc`

Basic Queries

Atlas Statistics

```
stats = statistics(csqli)
println("Nodes: ", stats[:n_nodes])
println("Edges: ", stats[:n_edges])
println("Support records: ", stats[:n_support])
println("Score range: ", round(stats[:min_score]; digits=2), " - ", round(stats[:max_score]; digits=2))

Nodes: 7
Edges: 6
Support records: 6
Score range: 0.8 - 0.95
```

Causal Backbone

The **backbone** returns the highest-scoring causal relationships:

```
bb = backbone(csqli; limit=10)
for row in bb
    println(" $(row.src) -[$(row.rel_type)]→ $(row.dst) (score=$(round(row.score_sum; digits=2)))")
end

transmission rate -[INCREASES]→ outbreak severity (score=0.95)
pathogen transmissibility -[INCREASES]→ transmission rate (score=0.92)
vaccination coverage -[INCREASES]→ population immunity (score=0.91)
contact rate -[INCREASES]→ transmission rate (score=0.87)
vaccination coverage -[REDUCES]→ host susceptibility (score=0.85)
host susceptibility -[INCREASES]→ transmission rate (score=0.8)
```

Causal Hubs

Hubs are the most influential concepts — nodes with the highest outgoing score mass:

```
hubs = causal_hubs(csqli; limit=5)
for h in hubs
    println(" $(h.concept): out_mass=$(round(h.out_mass; digits=2)), edges=$(h.n_edges)")
end

vaccination coverage: out_mass=1.76, edges=2
transmission rate: out_mass=0.95, edges=1
pathogen transmissibility: out_mass=0.92, edges=1
```

```

contact rate: out_mass=0.87, edges=1
host susceptibility: out_mass=0.8, edges=1

```

Downstream Effects

What does a concept *cause*?

```

effects = effects_of(csql, "vaccination")
for e in effects
    println("  vaccination coverage -[$(e.rel_type)]→ $(e.dst)  (score=$(round(e.score_sum; digits=2)))
end

vaccination coverage -[INCREASES]→ population immunity  (score=0.91)
vaccination coverage -[REDUCES]→ host susceptibility  (score=0.85)

```

Upstream Causes

What *causes* a concept?

```

causes = causes_of(csql, "transmission rate")
for c in causes
    println("  $(c.src) -[$(c.rel_type)]→ transmission rate  (score=$(round(c.score_sum; digits=2)))
end

pathogen transmissibility -[INCREASES]→ transmission rate  (score=0.92)
contact rate -[INCREASES]→ transmission rate  (score=0.87)
host susceptibility -[INCREASES]→ transmission rate  (score=0.8)

```

Custom SQL Queries

You can run arbitrary SQL against the underlying tables:

```

rows = custom_query(csql, """
    SELECT n.label_canon, n.deg_out, n.deg_in
    FROM atlas_nodes n
    ORDER BY n.deg_out + n.deg_in DESC
""")
for r in rows
    println("  $(r.label_canon): out=$(r.deg_out), in=$(r.deg_in)")
end

transmission rate: out=1, in=3
vaccination coverage: out=2, in=0
host susceptibility: out=1, in=1
pathogen transmissibility: out=1, in=0
contact rate: out=1, in=0
population immunity: out=0, in=1
outbreak severity: out=0, in=1

```

Summary

In this vignette you learned how to:

- Create an **AtlasBuilder** and add causal triples
- Build a CSQL database with **connect_csql()** and **build!()**
- Query the backbone, hubs, effects, and causes
- Run custom SQL queries

Next: Querying in Depth covers the full query API including multi-hop causal paths and feedback loop detection.